

Funciones y clases: El camino a la modularización

Santiago Gallego

2026-05-07

Contenido

0.1	Ejercicio 1 El cazador de coordenadas (Funciones y abstracción)	2
0.1.1	python	2
0.1.2	R	3
0.1.3	python	3
0.1.4	R	3
0.1.5	python	3
0.1.6	R	4
0.2	Ejercicio 2: Modelando una ruta espacial (Clases e interacción)	4
0.2.1	python	4
0.2.2	R	5
0.3	Preguntas	7
0.3.1	Sobre el Ejercicio 1 (Regla DRY y Pureza):	7
0.3.2	Sobre el ejercicio 2 (Paradigmas de POO):	7
0.3.3	Pregunta General (Eficiencia y Motores):	8

0.1 Ejercicio 1 El cazador de coordenadas (Funciones y abstracción)

En geomática, a menudo necesitamos saber cuál es la “caja delimitadora” (Bounding Box) de un conjunto de puntos para encuadrar un mapa.

1. Escribe una función pura llamada `calcular_bbox` que reciba una lista de múltiples coordenadas (latitud, longitud).
- La función debe recorrer la lista internamente y devolver cuatro valores: la latitud mínima, la latitud máxima, la longitud mínima y la longitud máxima.

0.1.1 python

```
def calcular_bbox(puntos):  
    """  
    Calcula el Bounding Box (Caja delimitadora) de una lista de puntos.  
    """  
    # Extraemos todas las latitudes y longitudes en listas separadas  
    # zip(*puntos) es un truco avanzado para "descomprimir" las columnas  
    lats, lons = zip(*puntos) # Desagrupa la lista de tuplas en dos secuencias: una de lats y una de lons  
  
    lat_min = min(lats) # Busca el valor de latitud más pequeño  
    lat_max = max(lats) # Busca el valor de latitud más grande
```

```

lon_min = min(lons) # Busca el valor de longitud más pequeño
lon_max = max(lons) # Busca el valor de longitud más grande

return lat_min, lat_max, lon_min, lon_max # Retorna los 4 límites como una tupla

```

0.1.2 R

```

#' Calcular Bounding Box
calcular_bbox <- function(puntos) {
  # Convertimos la lista de puntos en una matriz para fácil acceso por columnas
  matriz_puntos <- do.call(rbind, puntos) # Une los vectores de la lista por filas en una matriz

  lat_min <- min(matriz_puntos[,1]) # Extrae el mínimo de la primera columna (latitudes)
  lat_max <- max(matriz_puntos[,1]) # Extrae el máximo de la primera columna
  lon_min <- min(matriz_puntos[,2]) # Extrae el mínimo de la segunda columna (longitudes)
  lon_max <- max(matriz_puntos[,2]) # Extrae el máximo de la segunda columna

  # Devolvemos una lista nombrada para claridad
  return(list(
    lat_min = lat_min,
    lat_max = lat_max,
    lon_min = lon_min,
    lon_max = lon_max
  )) # Retorna un objeto de lista con etiquetas para cada valor
}

```

0.1.3 python

```

# Prueba del código
ruta = [(11.12, -74.03), (11.15, -74.05), (11.10, -74.01)] # Definimos una lista de coordenadas
b_box = calcular_bbox(ruta) # Llamamos a la función y guardamos el resultado

```

0.1.4 R

```

# Prueba del código
ruta_puntos <- list(c(11.12, -74.03), c(11.15, -74.05), c(11.10, -74.01)) # Definimos una lista
bbox_resultado <- calcular_bbox(ruta_puntos) # Ejecutamos la función con la lista de puntos

```

0.1.5 python

```

# Imprimimos los límites usando indexación de la tupla b_box
print(f"BBOX: Lat({b_box[0]}, {b_box[1]}) | Lon({b_box[2]}, {b_box[3]})")

```

BBOX: Lat(11.1, 11.15) | Lon(-74.05, -74.01)

0.1.6 R

```
cat("BBOX calculada:\n") # Imprime encabezado
```

BBOX calculada:

```
print(bbox_resultado)      # Muestra la lista con los 4 valores calculados
```

```
$lat_min  
[1] 11.1
```

```
$lat_max  
[1] 11.15
```

```
$lon_min  
[1] -74.05
```

```
$lon_max  
[1] -74.01
```

0.2 Ejercicio 2: Modelando una ruta espacial (Clases e interacción)

Vamos a llevar los objetos al siguiente nivel haciendo que interactúen entre sí. Toma el código base del PuntoEspacial que desarrollamos en este capítulo y crea una nueva estructura superior:

0.2.1 python

```
import math # Importamos librería para operaciones matemáticas  
  
class PuntoEspacial: # Definición de la clase base para un punto  
    def __init__(self, lat, lon):  
        self.lat = lat # Atributo de latitud  
        self.lon = lon # Atributo de longitud  
  
    def distancia_hacia(self, otro_punto): # Método para calcular distancia a otro objeto punto  
        # Distancia Euclidiana simple para el ejercicio  
        return math.sqrt((self.lat - otro_punto.lat)**2 + (self.lon - otro_punto.lon)**2)  
  
class RutaGeografica: # Clase contenedora para manejar una serie de puntos  
    def __init__(self, nombre):
```

```

self.nombre = nombre # Nombre de la ruta
self.puntos = [] # Lista vacía que almacenará objetos de tipo PuntoEspacial

def agregar_punto(self, punto): # Método para añadir puntos con validación
    if isinstance(punto, PuntoEspacial): # Verifica que el objeto sea de la clase correcta
        self.puntos.append(punto) # Lo añade a la lista interna
    else:
        print("Error: Solo se pueden agregar objetos PuntoEspacial")

def medir_distancia_total(self): # Método para calcular la longitud de toda la ruta
    distancia_total = 0 # Acumulador de distancia
    if len(self.puntos) < 2: # Si hay menos de 2 puntos, no hay tramos para medir
        return 0.0

    # Iteramos para medir tramos (punto A hacia punto B)
    for i in range(len(self.puntos) - 1): # Recorre hasta el penúltimo elemento
        tramo = self.puntos[i].distancia_hacia(self.puntos[i+1]) # Calcula distancia entre
        distancia_total += tramo # Suma el tramo al total
    return distancia_total # Retorna la suma acumulada

# --- PRUEBA ---
mi_ruta = RutaGeografica("Ruta de los Andes") # Instanciamos la clase ruta
mi_ruta.agregar_punto(PuntoEspacial(4.6, -74.0)) # Creamos punto Bogotá y lo agregamos
mi_ruta.agregar_punto(PuntoEspacial(3.4, -76.5)) # Creamos punto Cali y lo agregamos

# Imprimimos el reporte final con el nombre de la ruta y la distancia calculada
print(f"Distancia total en {mi_ruta.nombre}: {mi_ruta.medir_distancia_total():.2f} unidades")

```

Distancia total en Ruta de los Andes: 2.77 unidades

0.2.2 R

```

# --- CLASE PUNTO ESPACIAL (Usando sistema S3) ---
nuevo_punto_espacial <- function(lat, lon) { # Constructor del objeto
    objeto <- list(lat = lat, lon = lon) # Define estructura de lista
    class(objeto) <- "PuntoEspacial" # Asigna etiqueta de clase
    return(objeto)
}

# Definición de la función genérica
distancia_hacia <- function(obj1, obj2) UseMethod("distancia_hacia")

# Implementación del método específico para la clase PuntoEspacial
distancia_hacia.PuntoEspacial <- function(obj1, obj2) {
    sqrt((obj1$lat - obj2$lat)^2 + (obj1$lon - obj2$lon)^2) # Fórmula de distancia
}

```

```

}

# --- CLASE RUTA GEOGRÁFICA ---
nueva_ruta_geografica <- function(nombre) { # Constructor
  objeto <- list(
    nombre = nombre,
    puntos = list() # Lista vacía para almacenar los objetos PuntoEspacial
  )
  class(objeto) <- "RutaGeografica" # Asigna etiqueta de clase S3
  return(objeto)
}

# Función genérica para agregar puntos
agregar_punto <- function(ruta, punto) UseMethod("agregar_punto")

# Implementación del método para la clase RutaGeografica
agregar_punto.RutaGeografica <- function(ruta, punto) {
  if (class(punto) == "PuntoEspacial") { # Validación de tipo de objeto
    ruta$puntos[[length(ruta$puntos) + 1]] <- punto # Inserta el punto al final de la lista
    return(ruta) # En R (inmutable) es obligatorio devolver el objeto modificado
  } else {
    stop("El objeto no es de clase PuntoEspacial") # Lanza error si no es el tipo correcto
  }
}

# Función genérica para medir distancia
medir_distancia_total <- function(ruta) UseMethod("medir_distancia_total")

# Implementación del método de cálculo para RutaGeografica
medir_distancia_total.RutaGeografica <- function(ruta) {
  total <- 0 # Inicializa acumulador
  n <- length(ruta$puntos) # Cuenta cuántos puntos hay

  if (n < 2) return(0) # Si hay un punto o ninguno, la distancia es cero

  for (i in 1:(n - 1)) { # Recorre los tramos entre puntos consecutivos
    total <- total + distancia_hacia(ruta$puntos[[i]], ruta$puntos[[i+1]]) # Suma tramos
  }
  return(total) # Retorna el gran total
}

# --- PRUEBA ---
mi_ruta <- nueva_ruta_geografica("Ruta de los Andes") # Crea instancia de ruta
mi_ruta <- agregar_punto(mi_ruta, nuevo_punto_espacial(4.6, -74.0)) # Añade Bogotá y actualiza
mi_ruta <- agregar_punto(mi_ruta, nuevo_punto_espacial(3.4, -76.5)) # Añade Cali y actualiza va

cat("Ruta:", mi_ruta$nombre, "\n") # Imprime el nombre de la ruta

```

Ruta: Ruta de los Andes

```
cat("Distancia total:", medir_distancia_total(mi_ruta), "\n") # Imprime la distancia total
```

Distancia total: 2.773085

0.3 Preguntas

0.3.1 Sobre el Ejercicio 1 (Regla DRY y Pureza):

Explica con tus propias palabras qué es la regla DRY (Don't Repeat Yourself). ¿Por qué es un riesgo para la mantenibilidad de un proyecto copiar y pegar la gran fórmula matemática de Haversine en múltiples partes de tu código en lugar de usar la función que creaste?

La regla DRY (Don't Repeat Yourself) y la Pureza La regla DRY es un principio de diseño que dicta que cada pieza de conocimiento o lógica debe tener una representación única y autorizada dentro de un sistema.

El riesgo de copiar y pegar (Copy-Paste) Si se copia la fórmula de Haversine (que es bastante compleja debido a los senos, cosenos y el radio de la Tierra) en cinco lugares diferentes del código:

1. Mantenibilidad: Si descubro que usé un radio de la Tierra incorrecto (ej. 6371 km vs 6378 km), tendría que buscar y corregir el error en cinco sitios distintos. Es casi seguro que se olvidará uno, generando datos inconsistentes.
2. Legibilidad: El código se vuelve “ruidoso”. En lugar de leer `calcular_distancia()`, el programador tiene que descifrar trucos de álgebra cada vez que revisa el flujo principal.
3. Depuración: Si una función es pura y está centralizada, se puedes probar por separado (Unit Testing) y estar 100% seguro de que funciona antes de integrarla.

0.3.2 Sobre el ejercicio 2 (Paradigmas de POO):

Si resolviste (o resolvieras) este ejercicio en Python, ¿dónde “viven” físicamente los métodos analíticos como `medir_distancia_total()`? Contrasta esta filosofía con el enfoque del Sistema S3 de R. Usa la analogía del “Molde” vs el “Post-it” vista en clase para explicarlo.

Paradigmas de POO: El “Molde” vs. el “Post-it” Esta es la diferencia fundamental entre la Programación Orientada a Objetos (Python) y el Sistema S3 (R).

Python: El “Molde” de Hierro (Encapsulamiento) En Python, el método `medir_distancia_total()` vive dentro del molde de hierro (la clase).

La filosofía: Cuando usas el molde para crear una arepa (instancia), el diseño de los bordes y la forma ya están grabados en el hierro. No puedes separar la “acción de dar forma” de la arepa misma.

En la memoria: Físicamente, el método está definido en el objeto clase. Cuando llamas a `mi_ruta.medir_distancia_total()`, Python mira el molde de esa arepa específica y ejecuta la

instrucción que está allí guardada. El objeto “sabe” qué hacer porque sus instrucciones son parte de su naturaleza desde que salió del molde.

R: El “Post-it” (Despacho S3) En el Sistema S3 de R, la filosofía es radicalmente distinta: el método vive afuera, en un estante de herramientas general.

La filosofía: Aquí no hay un molde de hierro que marque la masa. Tú tienes una “masa de datos” (una lista) y le pegas un Post-it que dice “RutaGeografica”.

En la memoria: El método `medir_distancia_total` es una función independiente que flota en tu ambiente de trabajo. Cuando tú le lanzas el objeto a esa función, ella se fija en el color del Post-it. Si el Post-it dice “RutaGeografica”, busca en su manual de instrucciones qué debe hacer con ese tipo de objeto.

El contraste: En R, el objeto es “tonto”; no sabe hacer nada por sí mismo. Es la función externa la que decide cómo tratarlo basándose en la etiqueta (clase) que tiene pegada.

0.3.3 Pregunta General (Eficiencia y Motores):

Imagina que tu RutaGeografica tiene 5 millones de puntos recolectados por un GPS y necesitas calcular la distancia total. Si usarás Julia, explica brevemente cómo la combinación del Despacho Múltiple y la Compilación JIT lograría ejecutar este cálculo masivo mucho más rápido que el intérprete estándar de R.

Para procesar 5 millones de puntos, Julia elimina el “overhead” o carga administrativa que frena a R:

Compilación JIT (Just-In-Time): A diferencia de R, que interpreta el código en tiempo de ejecución (línea por línea), Julia compila el código a lenguaje de máquina optimizado antes de ejecutarlo. Esto permite que el cálculo de los 5 millones de puntos ocurra a velocidades cercanas a C++.

Despacho Múltiple: Julia genera versiones especializadas de la función para los tipos de datos exactos que se están usando. Al saber que los datos son, por ejemplo, `Float64`, el compilador puede implementar optimizaciones de hardware como instrucciones SIMD, permitiendo que el procesador realice múltiples operaciones matemáticas en un solo ciclo de reloj, algo que el intérprete estándar de R, al ser monohilo y de tipado dinámico estricto, no puede optimizar de forma tan agresiva.